CO1 ASSESSMENT 2

C Programming Assignment

Student Name	ROHITH V
Register Number	192312490
Course Code & Name	CSA0208, C programming
Slot	A
Assignment / Task Title	Reliable Billing Program for a Rural Electricity Co-operative
GitHub Repository URL	https://github.com/rohithvenkat26-ux/CSA0802A--C-Programming-.git
Submission Date	27-10-2026

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

TASK 1 — Organise the Program with Header Files and Conditional Compilation

Task / Problem Statement

Organise billing constants and function prototypes in a header file, use include guards, and enable debugging output only when compiled with -DDEBUG.

Aim

To structure the C billing program using a header file and conditional compilation.

Concept / Theory

```
Include guards prevent repeated inclusion of a header. #ifdef DEBUG includes diagnostic output only when DEBUG is defined during compilatic
```

Procedure / Algorithm

Create billing.h and place constants and the function prototype in it.

```
Add an include guard using #ifndef, #define, and #endif.
```

Include billing.h from the source file.

Place diagnostic output inside #ifdef DEBUG and #endif.

Compile and run normal and debug builds using GCC.

Program / Source Code

```
/* billing.h */
#ifndef BILLING_H
#define BILLING_H
#define FIXED_CHARGE_PAISE 5000LL
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE2_PAISE 225
#define RATE3_PAISE 450
long long compute_bill(int units);
#endif

/* task1.c */
#include <stdio.h>
#include "billing.h"

long long compute_bill(int units)
{
    long long paise = FIXED_CHARGE_PAISE;
    if (units < 0) return -1;
    if (units > SLAB2_UNITS) {
        paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS)
        paise += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    return paise;
}
int main(void)
{
    int units = 150;
    long long paise = compute_bill(units);
#ifdef DEBUG
    fprintf(stderr, "[DEBUG] units=%d, paise=%lld\\n", units, paise);
#endif
    printf("Units: %d\\nBill: Rs %lld.%02lld\\n",
           units, paise / 100, paise % 100);
    return 0;
}
```

Output Screenshot

Expected console output:

Units: 150\nBill: Rs 162.50

Analysis / Discussion

Normal compilation excludes the debug statement. Compilation with -DDEBUG includes the diagnostic line. A run-time debug flag needs a test each time the statement is reached and leaves the debug code in the executable; #ifdef DEBUG removes it from a normal build. Both approaches have O(1) overhead per reached debug statement when enabled, but compile-time exclusion has no run-time test.

Result

The program structure uses a guarded header and a conditional debug mode.

TASK 2 — Read and Print Data Correctly

Task / Problem Statement

Read previous and current 16-bit meter readings safely, calculate units including meter rollover, and print the bill accurately in rupees and paise.

Aim

To use correct scanf/printf format specifiers and validate user input.

Concept / Theory

```
Use %u for unsigned int and %lld for long long. Check scanf's return value before using input. Store money as integer paise and print rupee
```

Procedure / Algorithm

Read the previous and current readings using scanf.

Reject input unless scanf reads both values successfully.

Check that each reading is between 0 and 65535.

Calculate consumption using modulo 65536.

Calculate the slab bill in paise and print two decimal places.

Program / Source Code

```
#include <stdio.h>
#define METER_MODULUS 65536U
#define FIXED_CHARGE_PAISE 5000LL
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL

unsigned int consumption(unsigned int previous, unsigned int current)
{
    return (current + METER_MODULUS - previous) % METER_MODULUS;
}
long long compute_bill(unsigned int units)
{
    long long paise = FIXED_CHARGE_PAISE;
    if (units > 200U) {
        paise += (long long)(units - 200U) * RATE3_PAISE;
        units = 200U;
    }
    if (units > 100U)
        paise += (long long)(units - 100U) * RATE2_PAISE;
    return paise;
}
int main(void)
{
    unsigned int previous, current, units;
    long long bill;
    printf("Enter previous and current readings: ");
    if (scanf("%u %u", &previous, &current) != 2) {
        printf("Invalid input.\\n");
        return 1;
    }
    if (previous > 65535U || current > 65535U) {
        printf("Readings must be 0-65535.\\n");
        return 1;
    }
    units = consumption(previous, current);
    bill = compute_bill(units);
    printf("Units consumed: %u\\n", units);
    printf("Total bill: Rs %lld.%02lld\\n", bill / 100, bill % 100);
    return 0;
}
```

Output Screenshot

Example input 1200 1530 gives 330 units and Rs 860.00.

Analysis / Discussion

```
Using a mismatched format specifier can cause undefined behaviour. scanf return-value checking detects failed conversions. An alternative i
```

Result

The input is checked and the bill is printed with two paise digits.

TASK 3 — Control Type Conversion and Precision

Task / Problem Statement

Identify type conversions in billing calculations and prevent incorrect totals caused by integer division, unsafe casts, or floating-point precision.

Aim

To analyse C arithmetic conversions and use integer paise for exact billing.

Concept / Theory

Integer division discards fractional parts. Explicit casts change expression types, but converting a value outside int's range to int is implementation-defined. Floating-point values cannot represent all decimal fractions exactly; integer paise avoids decimal rounding during billing.

Procedure / Algorithm

Identify expressions that combine integer and floating-point types.

Compare integer division with division after casting to double.

```
Test a large long long value cast to int.
```

Represent money in integer paise.

Apply one documented rounding rule if fractional paise arise.

Program / Source Code

```
#include <stdio.h>
int main(void)
{
    int units = 250;
    double rate = 2.25;
    long long large = 3000000000LL;

    printf("250 / 100 = %d\\n", units / 100);
    printf("250 * 225 / 100 = %d\\n", units * 225 / 100);
    printf("(double)250 / 100 = %.2f\\n", (double)units / 100);
    printf("(int)large = implementation-defined if out of range\\n");
    printf("units * rate = %.2f\\n", units * rate);
    printf("integer paise = %lld\\n", (long long)units * 225LL);
    return 0;
}
```

Output Screenshot

Key expected results: 250/100 = 2; 250*225/100 = 562; (double)250/100 = 2.50; units*rate = 562.50; integer paise = 56250.

Analysis / Discussion

Integer division truncates the fraction, so 250/100 is 2. Using integer paise, 250 units at 225 paise per unit equals exactly 56250 paise. Both integer-paise and floating-point approaches take O(1) time per bill and O(n) time for n bills; integer paise uses O(1) extra space for a running total and avoids decimal representation errors when the integer type does not overflow.

Result

The examples demonstrate type-conversion risks and support integer paise for exact totals.

TASK 4 — Trace the Compilation Process

Task / Problem Statement

Trace the billing program through preprocessing, assembly generation, object-file generation, and linking, and distinguish compile-time, link-time, and run-time errors.

Aim

To understand the output of each GCC compilation stage and identify warnings.

Concept / Theory

gcc -E produces preprocessed source, gcc -S produces assembly, gcc -c produces an object file, and the final link step creates an executable. The preprocessor expands included headers and replaces macros.

Procedure / Algorithm

Save the source as task4.c.

Run gcc -E task4.c -o task4.i.

Run gcc -S task4.c -o task4.s.

Run gcc -Wall -Wextra -c task4.c -o task4.o.

Link the object file, execute it, and inspect any warnings.

Program / Source Code

```
#include <stdio.h>
#define FIXED_CHARGE_PAISE 5000LL
int main(void)
{
    long long paise = FIXED_CHARGE_PAISE;
    printf("Bill: Rs %lld.%02lld\\n", paise / 100, paise % 100);
    return 0;
}
```

Output Screenshot

Expected executable output:

Bill: Rs 50.00

Analysis / Discussion

A compile-time error can be caused by a missing semicolon. A link-time error occurs when a referenced function has no definition. A run-time error occurs during execution, such as division by zero. -Wall -Wextra can reveal unused variables or format mismatches. In a multi-file program, only changed source files normally need recompilation before relinking; object files use extra disk space but can reduce rebuild time.

Result

The compilation stages and the main categories of errors are identified.